

Hybrid-Grid: A Distributed Compilation System with Contextual-Bandit Task Scheduling on Heterogeneous Worker Clusters

Le Duc Hieu^a, Nguyen Trung Kien^a, Nguyen Trong Khanh^a

^a*Posts and Telecommunications Institute of Technology, Hanoi, Vietnam*

Abstract

This paper presents Hybrid-Grid, an open-source distributed compilation system for C/C++ on heterogeneous worker clusters, with online task scheduling as its central research focus. Existing distributed build systems rely on static heuristics — round-robin, least-loaded, or capability scoring — and never learn from execution outcomes, even though our measurements show that compile times on heterogeneous clusters exhibit extreme variance (a P99/P50 ratio of approximately 29×). To exploit this signal, we model the scheduling decision as a contextual bandit problem and implement and compare six schedulers inside a single real system: three heuristics (LeastLoaded, Power-of-Two-Choices, HEFT) and three online learners (ϵ -greedy, LinUCB, and Hybrid-LinUCB — our proposed variant, which augments the UCB score with a heuristic warm-start phase and a real-time load-penalty term). Experiments on a CPython compilation workload (~293 tasks per build) over a 5-worker heterogeneous Docker cluster, under a randomized complete block design with 10 repetitions and paired statistical testing (one-sided Wilcoxon signed-rank, Holm–Bonferroni correction, Cliff’s delta effect sizes), yield a two-sided result: Hybrid-LinUCB significantly outperforms the weaker learners (5.26 s faster than P2C and 6.80 s faster than pure LinUCB, $p = 0.003$, $|d| = 1.0$) but only ties the LeastLoaded heuristic ($p = 0.65$) — an honest negative result reinforced by a warm-bandit ablation demonstrating that the tie is intrinsic to the stationary workload rather than a cold-start artifact. An equally important methodological contribution: the rigorous protocol detected and eliminated a false positive (an apparent 7.9% advantage at $p = 0.0001$) that a simpler experimental design had produced through run-order confounding. These results delineate the conditions under which learned scheduling delivers real value in distributed build systems, identifying cache affinity and non-stationary environments as the most promising directions.

1. Introduction

Distributed compilation is an everyday reality of modern software engineering. Large C/C++ projects such as LLVM, Chromium, or the Linux kernel contain tens of thousands of translation units; a clean build on a single machine can take hours. Systems such as distcc [15], Bazel Remote Build Execution, and Incredibuild address this by pushing compilation tasks onto a pool of remote workers. That pool is inherently *heterogeneous*: a developer’s laptop, a Linux CI runner, an ARM cloud node, a dedicated build machine — differing in CPU cores, memory, instruction-set architecture, and instantaneous load.

Theoretically, assigning tasks to heterogeneous machines to minimize makespan is the $R||C_{\max}$ (unrelated parallel machines) problem: NP-hard, inapproximable below a factor of $3/2$ unless $P = NP$, with the best known approximation ratio being 2 [1]. In the online setting — where tasks arrive sequentially and must be dispatched immediately — the situation is harder still: Graham’s list scheduling [2] achieves a competitive ratio of $2 - 1/m$ on identical machines, but no tight universal competitive bound exists for unrelated machines. Production systems therefore rely on heuristics: round-robin, least-loaded, or Power-of-Two-Choices (P2C) [3]. What these heuristics share is that they decide from queue lengths or static capability scores and *never learn* from observed outcomes.

Our measurements on a CPython build (~293 compilation tasks per build) over a 5-worker heterogeneous cluster reveal two striking phenomena: (i) the ratio between the P99 and P50 compile-time percentiles reaches **29×**, and (ii) a single “best” worker absorbs **33% of all dispatches** under both static heuristics. High variance combined with concentrated dispatch suggests that a scheduler that *observes compile outcomes and adjusts future decisions* — an online learner — could improve makespan and tail latency.

However, published reinforcement-learning (RL) approaches to scheduling (Decima [12], DeepRM [11]) require simulator pre-training — unavailable for build systems with real compiler binaries and noisy hardware. The *contextual multi-armed bandit* family [7, 8] is a less-explored alternative: each scheduling decision is a one-step problem, learned online from observed rewards, with theoretical regret bounds of order $O(\sqrt{T})$ under linear-payoff assumptions [10]. The most studied algorithm in this family is LinUCB [9].

In this paper we build a complete distributed compilation system (Hybrid-Grid) and use it as an experimental platform to answer the question: *can a contextual bandit beat static heuristics at scheduling real distributed compilation?* The main contributions of this study are:

- **C1.** A complete open-source distributed build system in Go: the hgbuild CLI as a drop-in gcc/clang replacement, the hg-coord coordinator with worker registry, per-

worker circuit breakers, and a real-time dashboard, and the `hg-worker` executor supporting native and Docker cross-compilation; gRPC/HTTP2 communication with optional TLS/mTLS, zero-config worker discovery via mDNS, an xxhash-based content-addressable cache, and full observability (Prometheus, OpenTelemetry).

- **C2.** Six schedulers implemented behind a unified `LearningScheduler` interface supporting online reward feedback, including **Hybrid-LinUCB** — our proposed hybrid variant that adds a passively-learning warm-start window and a real-time load-penalty term $\lambda \cdot \text{LoadRatio}$ to the UCB score, addressing two observed failure modes of the pure bandit (cold start and delayed feedback).
- **C3.** A measurement pipeline emitting one 27-field JSON Lines record per dispatched task — covering worker context at decision time, latency decomposition, and learner introspection — enabling reproducible offline analysis.
- **C4.** A confound-controlled empirical evaluation (randomized complete block design, 10 repetitions, paired tests with multiple-comparison correction) plus a warm-bandit ablation, yielding an honest negative result — the bandit ties the strongest heuristic on a stationary workload — and a methodological lesson: the rigorous design caught a false positive produced by a careless one.

The remainder of this paper is structured as follows. Section 2 reviews related work. Section 3 formulates the online scheduling problem under the contextual-bandit framework. Section 4 describes the system architecture and scheduling algorithms, in particular the Hybrid-LinUCB design. Section 5 details the experimental setup and statistical protocol. Section 6 reports quantitative results. Section 7 discusses key findings, practical implications, limitations, and threats to validity. Section 8 concludes.

2. Related Work

2.1. Heuristic Scheduling on Heterogeneous Machines

The $R||C_{\max}$ problem was proven NP-hard by Lenstra, Shmoys, and Tardos [1], with a $3/2$ inapproximability lower bound and a 2-approximation algorithm; the gap remains open after more than 35 years. In practice, Power-of-Two-Choices [3] is the standout heuristic: sampling two random servers and picking the less loaded one reduces the maximum load from $\Theta(\log n / \log \log n)$ to $\Theta(\log \log n)$ on homogeneous clusters. Sparrow [4] extends P2C with late binding for low-latency decentralized scheduling. HEFT [5] is the classical algorithm for task DAGs on heterogeneous machines, ranking tasks by upward rank and assigning by earliest finish time; we adapt HEFT to an online task stream using EWMA estimates of compile time. The common trait across this group: decisions are based on queue state or static capability, never on per-task outcomes.

2.2. Multi-Armed and Contextual Bandits

The classical bandit framework with ϵ -greedy and incremental sample-mean updates is systematized by Sutton and Barto [6]; UCB1 by Auer et al. [16] introduced optimism-under-uncertainty into arm selection. Slivkins [7] and Lattimore–Szepesvári [8] systematize regret theory. LinUCB [9] extends to the contextual setting: each arm maintains a linear model (A_a, b_a) and is selected by the score $\hat{\theta}_a^\top x + \alpha \sqrt{x^\top A_a^{-1} x}$. The $\tilde{O}(\sqrt{Td})$ regret bound is proven for the SupLinUCB variant [10]; notably, this bound does *not* directly apply to the LinUCB Algorithm 1 that practical systems (including ours) implement — a point we state honestly in Section 7.

2.3. Machine Learning for Systems Scheduling

Decima [12] uses a graph neural network policy with REINFORCE to schedule Spark jobs, reporting up to $1.5\times$ job-completion-time improvement, but requires thousands of training episodes in a simulator before deployment. DeepRM [11] was the earliest to apply policy gradients to resource management, likewise simulator-dependent. Quasar [13] uses collaborative filtering (supervised learning, with no exploration) to predict job performance per machine type. Resource Central [14] uses random forests to predict VM lifetimes in Azure — supervised, offline, deployed at production scale. The gap this work targets: *online learning, with no simulator, applied specifically to distributed compilation* — to our knowledge, no published work applies contextual bandits to this problem and characterizes the implementation pitfalls that arise from interaction with real compile-latency distributions.

2.4. Distributed Build Systems

Production build systems fall into three classes: *cache-first* systems (ccache, sccache, Bazel RBE) prioritize hash-based cache lookup and select workers by round-robin or least-loaded; *capability-scored* systems (distcc [15]) rank workers by static capability; *DAG-aware* systems (Bazel, Buck2) use the build graph to defer decisions but still schedule within each DAG layer by capability. None learn online. The closest published works use offline ML to predict task duration and then dispatch heuristically on those predictions — supervised-learning scheduling, not bandit learning.

3. Problem Definition

3.1. System Context

Hybrid-Grid splits a C/C++ build into hundreds of independent tasks (each preprocessed `.c/.cpp` file is one task) and distributes them to a heterogeneous worker cluster. The coordinator maintains a registry of live workers — each described by static features (CPU cores, memory, native architecture, OS) and live counters (active tasks, recent RPC latency) — and decides which task goes to which worker. This decision point has the largest influence on makespan and is the focus of this research.

3.2. Contextual-Bandit Formulation

At decision time t , the coordinator observes a context vector $x_{t,a} \in \mathbb{R}^d$ for each eligible worker a (after filtering out unhealthy workers, open circuit breakers, and workers at their parallelism cap), selects an action a_t , dispatches the task, and upon completion receives the scalar reward

$$r_t = -\log(1 + t_{\text{compile}}^{(t)}), \quad (1)$$

where $t_{\text{compile}}^{(t)}$ is the worker-reported compile time in milliseconds. The log transform is an engineering choice to compress the heavy tail of the distribution (P99/P50 $\approx 29\times$): without compression, a single 24-second outlier would dominate sample-mean updates. Failed tasks receive $r = -\log(1 + T_{\text{timeout}})$ — the worst case the system could observe — so that the learner automatically discounts persistently failing workers. We note honestly that this reward shape has no direct peer-reviewed source; the closest theoretically-motivated formulation is Decima’s time-integrated reward [12].

We choose the bandit framing over a full MDP for two reasons. First, compilation tasks are nearly independent — once queue depth is part of the context vector, the residual state coupling between consecutive decisions is small [7]. Second, an MDP would require attributing the build’s makespan back to individual dispatch decisions — a credit-assignment problem with sparse rewards over hundreds of steps, exactly the regime where policy-gradient methods need a simulator [11, 12] that we deliberately do not have.

4. System and Methods

4.1. Hybrid-Grid Architecture

The system comprises three main components communicating via gRPC over HTTP/2 (optional TLS/mTLS):

- **hgbuild (CLI)**: a drop-in replacement for gcc/clang that also wraps make/ninja. It preprocesses sources locally, hashes the preprocessor output with xxhash for cache lookup, submits tasks to the coordinator, and automatically *falls back* to local compilation when the coordinator is unavailable.
- **hg-coord (Coordinator)**: the central orchestrator, comprising the worker registry (gRPC handshake registration, 10-second heartbeats), the scheduler (Section 4.2), per-worker circuit breakers for fault isolation, Prometheus metrics, OpenTelemetry tracing, and a real-time WebSocket dashboard.
- **hg-worker (Worker)**: executes compilations in two modes — native (direct gcc/clang invocation) or Docker (cross-compilation via dockcross images); maintains a local content-addressable cache ($\sim 10\times$ speedup on hits); self-advertises via mDNS for zero-config discovery.

Beyond the central C/C++ path, the system also supports distributed Flutter Android builds and GCC/Clang-to-MSVC flag translation; these paths do not pass through the learning scheduler and are outside the scope of the evaluation.

Table 1: The 12-dimensional feature vector of (Hybrid-)LinUCB.

#	Feature	Normalization
0	bias	constant 1.0
1	log preprocessed source size	$\frac{\log(1+s)}{\log(1+4 \text{ MiB})}$, clip 1
2	target arch = x86_64	one-hot 0/1
3	target arch = arm64	one-hot 0/1
4	worker CPU cores	cores/16, clip 1
5	worker memory	mem/64 GiB, clip 1
6	native arch matches target	0/1
7	queue pressure	$\frac{\text{active}}{\text{max_parallel}}$, clip 1
8	recent RPC latency	ms/100, clip 1
9	task is C++	0/1
10	log raw source size	$\frac{\log(1+s_{\text{raw}})}{\log(1+1 \text{ MiB})}$, clip 1
11	worker success rate (Laplace)	$(s+1)/(s+f+2)$

4.2. Unified Scheduling Framework

The Scheduler interface exposes `Select(buildType, arch, clientOS)  $\rightarrow$  (Worker, error)`. The extended `LearningScheduler` interface adds `SelectWithDispatchInfo` (also returning the Q-value and an exploration flag for logging) and `RecordOutcome`. The coordinator’s `Compile()` handler uses a type assertion to feed rewards back only when a learner is configured — the heuristics require no modification. Six schedulers are selected via the `-scheduler` CLI flag: (1) **LeastLoaded**; (2) **P2C** [3]; (3) **HEFT** with EWMA compile-time estimates ($\alpha = 0.3$) [5]; (4) ϵ -**greedy** with per-worker sample means and $\epsilon = 0.1$ [6] — a *feature-blind* policy; (5) **LinUCB** (Section 4.3); (6) **Hybrid-LinUCB** (Section 4.4). All three learners share a single-candidate fast path: when only one eligible worker remains, it is returned immediately without matrix algebra — eliminating the 41% penalty observed for P2C’s filtering pipeline on 1-worker clusters.

4.3. The LinUCB Scheduler

Following Li et al. [9], for each worker a we maintain $A_a \in \mathbb{R}^{d \times d}$ initialized to I_d and $b_a \in \mathbb{R}^d$ initialized to $\mathbf{0}$. The current ridge-regression estimate is $\hat{\theta}_a = A_a^{-1}b_a$. At round t , the UCB score of each eligible worker is

$$p_{t,a} = \hat{\theta}_a^\top x_{t,a} + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}}, \quad (2)$$

we select $a_t = \arg \max_a p_{t,a}$, and upon reward arrival update $A_{a_t} \leftarrow A_{a_t} + x x^\top$, $b_{a_t} \leftarrow b_{a_t} + r x$.

Feature vector ($d = 12$). Table 1 lists the feature layout; all features are scaled to approximately $[0, 1]$ to keep $\|x\|$ bounded, following the linear-payoff convention of [10]. Two design decisions deserve mention. First, the initial design used a three-way one-hot for build type (CPP/Flutter/Unity); because the current `Compile()` path only emits CPP, these dimensions degenerated (the CPP column perfectly collinear with the bias, the other two permanently zero) and were removed. Second, a raw success-rate feature with an optimistic prior would equal exactly 1.0 for every worker on a fully-successful workload —

again collinear with the bias — so we apply Laplace smoothing to keep the column experience-dependent.

Sherman–Morrison incremental inverse. Each update is rank-1: $A_{\text{new}} = A_{\text{old}} + xx^T$. Re-inversion from scratch costs $O(d^3)$; the Sherman–Morrison formula [17] reduces this to $O(d^2)$:

$$A_{\text{new}}^{-1} = A_{\text{old}}^{-1} - \frac{A_{\text{old}}^{-1}xx^TA_{\text{old}}^{-1}}{1 + x^TA_{\text{old}}^{-1}x}. \quad (3)$$

A unit test compares the cached inverse against a fresh inversion after 50 random updates; the elementwise discrepancy stays below 10^{-6} .

Three implementation traps. The first LinUCB run ($\alpha = 1.0$) performed *worse than every baseline* (158 s on the 5-worker cluster, versus 94 s for P2C). An independent code review identified three bugs: (i) *target leakage* — the feature vector was reconstructed at update time using worker state *after* task completion rather than at dispatch; (ii) *multicollinearity* of the build-type one-hot as described above; (iii) *reward-scale mismatch* — the reward magnitude ($|r| \approx 7$) dwarfed the exploration bonus, driving the effective exploration rate down to 1.7%. After the fixes (caching the feature vector at Select keyed by TaskID, removing the degenerate dimensions, lowering α to 0.5), makespan recovered to 94 s — **the entire 64-second recovery came from implementation discipline, not algorithmic redesign.**

4.4. Hybrid-LinUCB: The Proposed Design

Observing pure LinUCB revealed two remaining failure modes: *cold start* — during the first dispatches, the estimates $\hat{\theta}_a$ carry no information and decisions are near-random; and *delayed feedback* — rewards arrive seconds later, and within that window the bandit cannot see that a worker is being flooded and keeps flooding it. Hybrid-LinUCB addresses both with two orthogonal mechanisms, configured via `-warm-start` (default $N = 100$) and `-load-penalty` (default $\lambda = 0.5$):

(a) Passively-learning warm-start. For the first N dispatches, decisions are delegated to the least-loaded heuristic, but the learner *still* computes and stores the feature vector of the selected worker; when the reward arrives, the Sherman–Morrison update proceeds normally. The bandit “watches and learns” while ceding decision authority — after N dispatches it takes over with matrices already warmed by real observations.

(b) Hybrid score with load penalty. After warm-start, the selection score becomes

$$Q_{\text{hybrid}} = \underbrace{\hat{\theta}_a^T x}_{\text{learned value}} + \underbrace{\alpha \sqrt{x^T A_a^{-1} x}}_{\text{exploration bonus}} - \underbrace{\lambda \cdot \text{LoadRatio}_a}_{\text{overload penalty}}. \quad (4)$$

LoadRatio is already feature $x[7]$, so in principle LinUCB could learn a negative weight for it on its own; the penalty term acts as a *manual prior* that uses real-time information (task counts current to the millisecond, as LeastLoaded does) to suppress worker-flooding *before* the model has accumulated enough data to discover it — the central “hybrid” argument of the design. We constrain $\lambda \in [0, 5]$ because an oversized λ lets the penalty dominate every learned estimate, degenerating

into LeastLoaded. With both parameters at zero, behavior is bit-identical to pure LinUCB — guaranteeing that benchmark comparisons run the same code path and differ only in configuration.

4.5. Measurement Pipeline

Every completed `Compile()` invocation emits one 27-field JSON Lines record: identity (task, build type, scheduler), worker context at dispatch (cores, RAM, architecture, active tasks, discovery source), task metadata (raw/preprocessed source size, target architecture), latency decomposition (queue/compile/RPC/total), outcome (success, exit code, cache hit), and learner introspection (Q-value at dispatch, exploration flag). Files load directly into pandas via `read_json(lines=True)`; the schema was validated at 100% conformance on 1,746 records from the preliminary measurement phase.

5. Experiments

5.1. Setup

Workload: a clean CPython build via `make` with `hgbuild` as the compiler driver — ~293 translation units per build (light load) and a ~371-unit variant (heavy load). **Cluster:** Docker Compose on a single host, with total CPU fixed at 4.0 cores divided unequally across 5 workers (0.5/0.6/0.8/1.0/1.1 cpu) via `cgroup` limits; 1-worker and 3-worker configurations serve the preliminary measurements. **Metrics:** makespan (wall-clock), per-worker dispatch distribution, and P50/P95/P99 compile-time percentiles.

5.2. Randomized Block Design and Statistical Analysis

Preliminary measurements showed that the single-host Docker platform has a substantial run-to-run noise floor; single-run comparisons are therefore inconclusive. More importantly, a “scheduler-major” layout (running all repetitions of one scheduler back-to-back) *entangles* scheduler identity with slow host drift (thermal, background load). We redesigned the experiment as a **randomized complete block design**: 10 rounds, each running all four schedulers exactly once in a per-round seeded shuffled order; with one discarded warm-up build, a barrier waiting for all 5/5 workers to register, a fixed cooldown between builds, sub-second timing, and compile-cache clearing before every build. Each round forms a statistical block, enabling **paired** tests: Friedman omnibus [20], one-sided Wilcoxon signed-rank [18], Holm–Bonferroni correction [19], Cliff’s delta effect sizes [21], and bootstrap confidence intervals.

6. Results

6.1. Preliminary Single-Run Measurements

Three observations from Table 2: (i) P2C improves 1.45–1.62× over LeastLoaded on heterogeneous clusters, matching the theoretical prediction [3]; (ii) feature-blind ϵ -greedy

Table 2: Makespan (s), single-run — context only, not a valid basis for comparative conclusions.

Config	LL	P2C	ε -g	LinUCB ^{bug}	LinUCB ^{fix}	HEFT
1w-4.0cpu	92	130	146	129	131	129
3w-hetero	123	85	142	103	108	135
5w-hetero	152	94	119	158	94	144

Table 3: Makespan (s), 10 blocks, 5-worker heterogeneous cluster, light workload (~293 tasks/build). Paired Wilcoxon + Holm vs. Hybrid-LinUCB.

Scheduler	Median	Mean	Std	vs. hybrid
LeastLoaded	40.24	40.38	1.02	tie ($p = 0.65$)
Hybrid-LinUCB	40.52	40.56	1.06	—
P2C	45.60	46.01	1.39	wins ($p = 0.003$)
LinUCB	47.48	47.11	1.45	wins ($p = 0.003$)

loses to *every* heuristic — the learner faithfully identifies “the worker with the best mean reward” and floods it, skewing load worse than LeastLoaded (top:bottom dispatch ratio 13.2:1 versus P2C’s 8.6:1); (iii) buggy LinUCB was the worst scheduler on 5 workers (158 s), recovering to 94 s after the three implementation fixes (Section 4.3) — with the exploration rate rising from 1.7% to 25.1% and load skew dropping from 97:1 to 13.9:1.

6.2. Headline Result: The Confound-Controlled Comparison

The Friedman omnibus gives $\chi^2 = 24.60$, $p = 2 \times 10^{-5}$: the schedulers differ significantly (Table 3). Hybrid-LinUCB beats P2C ($\Delta = -5.26$ s) and pure LinUCB ($\Delta = -6.80$ s) with maximal effect size ($|d| = 1.0$ — winning in all 10 of 10 blocks), but is **statistically indistinguishable from LeastLoaded** ($\Delta = +0.09$ s, $d = +0.10$). For P99 tail latency, Friedman gives $p = 0.169$ — no detectable difference among the four schedulers; the 2.3% tail advantage seen in the single run did not replicate.

A false positive caught. The first analysis of the same measurement effort, taken under the scheduler-major layout, showed Hybrid-LinUCB *beating* LeastLoaded by 7.9% at $p = 0.0001$. Once the run-order confound was removed by blocking, that advantage vanished entirely ($p = 0.65$): the apparent difference was an *artifact* of thermal/background drift — LeastLoaded had simply been measured while the host was in a different state. This is direct evidence that a rigorous experimental design can catch a false positive that a careless one produces.

Under heavier load (Table 4), LeastLoaded is fastest by median and the hybrid’s edge over P2C/LinUCB does not survive Holm correction. The hybrid’s large standard deviation (16.6) comes from exactly one outlier (one round at 101.93 s, roughly double the norm): a bad sequence of exploratory decisions during cold start exposes a **tail risk** of the bandit that static heuristics simply do not have.

6.3. Warm-Bandit Ablation: The Tie Is Intrinsic

The natural hypothesis explaining the tie: the benchmark restarts the coordinator on every build, so the bandit always starts cold. We tested this by keeping **one** coordinator alive

Table 4: Makespan (s) on the heavy workload (~371 tasks/build).

Scheduler	Median	Mean	Std	vs. hybrid
LeastLoaded	48.65	51.36	6.78	tie ($p = 0.82$)
Hybrid-LinUCB	50.64	55.42	16.60	—
P2C	53.48	55.22	5.08	$p_{\text{holm}} = 0.16$
LinUCB	60.20	59.45	6.35	$p_{\text{holm}} = 0.16$

across $K = 8$ sequential builds (the bandit’s state is an in-memory singleton, accumulating throughout), repeated over 6 independent sessions. The result refutes the hypothesis: the makespan trend over the build index is **flat** (Spearman $\rho = -0.079$, $p = 0.59$); build 8 is no faster than build 1 ($\Delta = +0.21$ s, $p = 0.42$); and the warmed bandit still ties LeastLoaded ($\Delta = -0.19$ s, $p = 0.50$). The explanation: each build contains ~293 tasks while the warm-start window is only 100 — the bandit converges *within a single build*; with a 12-dimensional feature space and a stationary workload, the matrices A_a saturate during the first build, and subsequent builds carry no new information. The tie is therefore not a cold-start consequence but intrinsic: on a stationary compilation workload with static workers, LeastLoaded is already near-optimal, and there is no hidden structure for the bandit to exploit beyond it.

6.4. Hyperparameter Sensitivity

Sweeping $\alpha \in \{0.1, 0.5, 1.0, 2.0\}$ on the 5-worker configuration (post-bugfix) yields makespans of 98/94/–/96 seconds — every value in the range is competitive with P2C, with differences inside the single-run noise band. The lesson for practitioners: the implementation-fix discipline matters far more than tuning α . The bandit’s computational cost is negligible: the 12-dimensional scoring step takes microseconds per worker, dwarfed by the gRPC round-trip of the dispatch itself.

7. Discussion

7.1. Key Findings

The two-sided outcome of this study can be summarized in three statements. *First*, static heuristics are surprisingly strong: LeastLoaded — no learning, no parameters — matches or beats every learner on a stationary compilation workload. *Second*, within the family of learners, context and hybridization deliver clear value: Hybrid-LinUCB wins decisively (10/10 blocks) against both P2C and pure LinUCB, showing that the warm-start and load penalty address exactly the two failure modes identified. *Third*, most of the bandit’s initial performance gap lay not in the algorithm but in implementation traps — target leakage, feature multicollinearity, reward-scale mismatch — errors that the algorithmic literature does not warn about and that only surface when running on a real system.

7.2. Practical Implications

For builders of distributed build systems, the results suggest: use LeastLoaded/P2C as the default on clean-build workloads;

invest in learned scheduling only when the environment carries structure the heuristics ignore. Our analysis identifies two such environments: (i) *cache affinity* — when a worker already holds a file’s artifact in its local cache, re-dispatching that file to the same worker turns a compilation into a near-instant cache hit; this signal correlates with dispatch-history features that Least-Loaded is entirely blind to, and only manifests in incremental-build scenarios (not the clean builds this study measures); (ii) *non-stationary environments* — workers degrading due to thermal throttling or background load, where an outcome-observing learner can react while a static capability heuristic cannot.

7.3. Limitations

(1) *One workload*: the entire evaluation uses CPython — a single C codebase; template-heavy C++ codebases (Boost, Eigen, LLVM) have heavier-tailed distributions where the relative ordering of schedulers may change. (2) *One topology*: the cluster runs on a single Docker host — symmetric network latency, zero clock skew, a shared filesystem cache. (3) *Linear realisability*: the regret bound of [10] requires $\mathbb{E}[r|x] = x^\top \theta^*$; compile time is non-linear in source size, and the log transform compresses but does not linearize — misspecification of magnitude ε inflates regret by an additive $O(\varepsilon \sqrt{T})$ [8]. (4) *Plain LinUCB versus SupLinUCB*: the $\tilde{O}(\sqrt{T}d)$ bound is proven for SupLinUCB; we implement the simpler LinUCB Algorithm 1, which has no proven bound. (5) *Drift*: no mechanism (sliding windows, change-point detection) is implemented to handle worker performance drift. (6) *No production-system comparison*: comparisons are restricted to the algorithms we implement, not the schedulers inside Bazel RBE or Incredibuild.

7.4. Threats to Validity

Internal: run-to-run noise of Docker on a single host is the main threat; the randomized block design with paired testing was chosen precisely to neutralize it, and the single-run tables are clearly labeled as non-conclusive context. *External*: one workload and one topology; moreover, all four schedulers in the headline evaluation ran on the same hardware configuration, so conclusions do not automatically extend to clusters of other scales. *Construct*: clean-build makespan is the primary metric; if the deployment goal is tail latency or incremental builds, the scheduler ranking could differ — our P99 data (statistically indistinguishable) underlines this.

8. Conclusion and Future Work

This paper presented Hybrid-Grid — a complete distributed compilation system in Go with six schedulers behind a unified learning interface — and a confound-controlled empirical study of contextual-bandit scheduling on heterogeneous clusters. The honest outcome is two-fold. On *results*: Hybrid-LinUCB (warm-start + load penalty) significantly outperforms the weaker learners but only ties the LeastLoaded heuristic on a stationary compilation workload — a tie shown to be intrinsic by the warm-bandit ablation, not a cold-start consequence. On *methodology*: the randomized block protocol with paired

testing detected and eliminated a false positive produced by run-order confounding — a lesson of independent value to the systems-measurement community.

Future work follows directly from the diagnosis of “when the learner wins”: (i) *cache-aware* scheduling on incremental-rebuild workloads — adding a “worker already holds this artifact” feature that the coordinator can derive from dispatch history without any new RPC; (ii) *drift* adaptation via change-point detection (CUSUM, Page–Hinkley) with localized resets of A_a, b_a ; (iii) a Decima-style reward $-(t_k - t_{k-1})J_k$ motivated by Little’s law in place of the purely-engineering log-latency form; (iv) extending the context vector once the Flutter/Unity build paths enter the learning flow. All source code, raw measurement logs, and reproduction scripts are released with the system.

Acknowledgment

[Add funding, institutional support, and advisor acknowledgments here.]

References

- [1] J. K. Lenstra, D. B. Shmoys, É. Tardos, Approximation algorithms for scheduling unrelated parallel machines, *Mathematical Programming* 46 (1990) 259–271. doi:10.1007/BF01585745.
- [2] R. L. Graham, Bounds on multiprocessing timing anomalies, *SIAM Journal on Applied Mathematics* 17 (2) (1969) 416–429.
- [3] M. Mitzenmacher, The power of two choices in randomized load balancing, *IEEE Transactions on Parallel and Distributed Systems* 12 (10) (2001) 1094–1104. doi:10.1109/71.963420.
- [4] K. Ousterhout, P. Wendell, M. Zaharia, I. Stoica, Sparrow: distributed, low latency scheduling, in: *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 69–84. doi:10.1145/2517349.2522716.
- [5] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Transactions on Parallel and Distributed Systems* 13 (3) (2002) 260–274.
- [6] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [7] A. Slivkins, Introduction to multi-armed bandits, *Foundations and Trends in Machine Learning* 12 (1–2) (2019) 1–286. arXiv:1904.07272.
- [8] T. Lattimore, C. Szepesvári, *Bandit Algorithms*, Cambridge University Press, 2020.

- [9] L. Li, W. Chu, J. Langford, R. E. Schapire, A contextual-bandit approach to personalized news article recommendation, in: *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 661–670. doi:10.1145/1772690.1772758.
- [10] W. Chu, L. Li, L. Reyzin, R. E. Schapire, Contextual bandits with linear payoff functions, in: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 208–214.
- [11] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016, pp. 50–56. doi:10.1145/3005745.3005750.
- [12] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, in: *Proceedings of ACM SIGCOMM*, 2019, pp. 270–288. doi:10.1145/3341302.3342080.
- [13] C. Delimitrou, C. Kozyrakis, Quasar: resource-efficient and QoS-aware cluster management, in: *Proceedings of ASPLOS*, 2014, pp. 127–144. doi:10.1145/2541940.2541941.
- [14] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, R. Bianchini, Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms, in: *Proceedings of SOSP*, 2017, pp. 153–167. doi:10.1145/3132747.3132772.
- [15] M. Pool, distcc: a fast, free distributed C/C++ compiler, 2004. <https://distcc.github.io/>.
- [16] P. Auer, N. Cesa-Bianchi, P. Fischer, Finite-time analysis of the multiarmed bandit problem, *Machine Learning* 47 (2–3) (2002) 235–256.
- [17] J. Sherman, W. J. Morrison, Adjustment of an inverse matrix corresponding to a change in one element of a given matrix, *The Annals of Mathematical Statistics* 21 (1) (1950) 124–127.
- [18] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [19] S. Holm, A simple sequentially rejective multiple test procedure, *Scandinavian Journal of Statistics* 6 (2) (1979) 65–70.
- [20] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, *Journal of the American Statistical Association* 32 (200) (1937) 675–701.
- [21] N. Cliff, Dominance statistics: ordinal analyses to answer ordinal questions, *Psychological Bulletin* 114 (3) (1993) 494–509.